InspectR — Glossary of Terms

Context: MSc thesis prototype (University of Oslo, in collaboration with SINTEF within DataPACT) for design-time compliance inspection of data-pipeline blueprints with Policy-as-Code (OPA/Rego).

Sirin Koca · Department of Informatics, University of Oslo · Rule set `tk-v2.0` · Draft for supervisor discussion, November 2026

How to read this glossary. Section 1 gives the operational definitions used in InspectR. They are taken from the prototype (`pages/method.py`, "Definitions"; `README.md`, "Terminology") and are definitions for this artefact, not general legal definitions. The "Grounding" column points to the literature or standard that the concept is based on or analogous to; it does not claim that the cited source uses identical wording. Sections 2–5 explain the background concepts (Compliance-by-design, Policy-as-Code, OPA, Rego) and the method for translating a regulatory requirement into an executable rule. Numbers in square brackets refer to the References.

Assurance boundary. InspectR checks what a blueprint declares against encoded interpretations of selected requirements. It does not establish legal compliance, completed assessments or approvals, or the runtime behaviour of controls. The encoded rules are the author's interpretations and have not yet been reviewed by SINTEF, Trondheim Kommune or legal experts.

1. InspectR terminology

The terms follow one chain: a **requirement** (natural language) is written as a **rule** (IF trigger THEN control); the rule reads **facts** from the **blueprint**; OPA evaluates the rule's decision table; the outcome is a **result** carrying one **status**.

```
Requirement (TK / GDPR, natural language)
  -> Rule        IF trigger THEN control
  -> Facts       blueprint fields the rule reads: true | false | undetermined
  -> Evaluation  OPA applies the rule's decision table to the facts
  -> Result      one status + reason + evidence + legal reference + remedy
```

1.1 Merged term table (alphabetical)

Origin: **M** = `pages/method.py` (preferred where both define a term), **R** = `README.md` only.

Term	Definition in InspectR	Origin	Grounding
Blueprint	Machine-readable description of a planned pipeline (in InspectR: a JSON document with typed nodes, connections and declared properties).	R	Design artefact in Design Science Research [22]
Compliance gate	A planned assessment or approval point in the pipeline design; declaring it does not establish enforcement.	R	Control objectives placed in process design [4], [5]
Control	The safeguard in the design that meets the obligation.	M	"Control" as a risk-modifying measure / safeguard [17], [18]

Term	Definition in InspectR	Origin	Grounding
Design-time	Checked on the blueprint before the pipeline is built or run.	M	Compliance-by-design, preventive vs. after-the-fact checking [4]; GDPR Art. 25(1) [1]
Evidence	The blueprint field read, its observed value and its expected value. (README: the blueprint observations and derived facts a result is based on.)	M	Accountability, ability to demonstrate compliance, GDPR Art. 5(2) [1]
Fact	A typed blueprint field a rule reads: true, false or undetermined.	M	Three-valued (Kleene) logic [15]
Fail-closed	Undetermined evidence never becomes PASS.	M	Principle of fail-safe defaults [16]
Machine-checkable	Expressed as a typed field the engine can read without a human.	M	Rules as Code [11]; isomorphism between rule and source [9]
Obligation	What the requirement demands once triggered.	M, R	Deontic modality (obligation, permission, prohibition) [6], [8]
Policy-as-Code (PaC)	Executable representation of policy conditions; in InspectR, Rego evaluated by OPA. See Section 3.	R	OPA documentation [12]
Requirement	What the law or the organisation demands, in words.	M	Regulatory requirements analysis [8]
Result	Status plus reason, evidence, legal reference and remedy.	M	SARIF `result` object [19]
Rule	A requirement written as IF trigger THEN control.	M	Conditional normative rules [5], [6], [7]
Status	The outcome of one rule for one blueprint (one of the six values in 1.2).	M	SARIF `kind` [19]; AWS Config [20]; Kyverno [21]
Trigger	The fact that makes the obligation apply.	M, R	Condition (antecedent) of a normative rule [6], [8]
Undetermined	No usable evidence for a fact: absent, null, or wrong type (not a JSON boolean).	M, R	Third truth value “unknown” [15]

1.2 Status values

Every rule is a complete decision table, so every input reaches exactly one status [23]. An undetermined fact never produces PASS.

Status	Meaning	Who acts next	Comparable value in standards / tools
PASS	The obligation applies and every required control is declared `true`. Not a compliance certificate.	Nobody (design level only)	SARIF `pass`; AWS Config `COMPLIANT`; Kyverno `pass`
FAIL	The obligation applies and a required control is declared `false`.	Designer: add the control	SARIF `fail`; AWS Config `NON_COMPLIANT`; Kyverno `fail`
UNKNOWN	A fact the rule needs is undetermined.	Designer or tool: supply the fact	SARIF `open`; AWS Config `INSUFFICIENT_DATA`
HUMAN	Everything checkable is declared; the final decision is a legal judgement.	Legal or domain expert	SARIF `review`
NOT_APPLICABLE	The trigger facts are declared `false`; the obligation does not apply.	Nobody	SARIF `notApplicable`; AWS Config `NOT_APPLICABLE`; Kyverno `skip`
ERROR	The rule could not be evaluated. Not a finding about the blueprint.	InspectR developer	Kyverno `error`; SARIF tool execution notification

Decision table shared by all rules (first matching row wins):

Trigger	Control	Status
undetermined	any	UNKNOWN
false	any	NOT_APPLICABLE
true	false	FAIL
true	undetermined	UNKNOWN
true	true	PASS (HUMAN for judgement rules)

2. Background concepts

Concept	Definition	Source
Compliance-by-design	Building compliance requirements into a system or process when it is designed, by modelling control objectives alongside the design, instead of detecting violations after deployment. Introduced for business-process compliance by Sadiq, Governatori and Namiri.	[4], [5]
Privacy by Design	Seven foundational principles requiring privacy to be embedded proactively into the design of systems, as the default setting.	[3]

Concept	Definition	Source
Data protection by design and by default	Legal obligation on the controller to implement appropriate technical and organisational measures, both when determining the means of processing and during processing, that implement the data-protection principles and integrate the necessary safeguards (GDPR Art. 25).	[1], [2]
Policy-as-Code (PaC)	Expressing policies (security, operational, regulatory) as machine-readable, version-controlled code that a policy engine evaluates automatically against structured input, so that decisions are repeatable, testable and reviewable like software.	[12]
Rules as Code	Producing an official, machine-consumable version of rules (law, regulation) alongside the natural-language text. Closely related to PaC, but from the rule-maker's side.	[11]
Open Policy Agent (OPA)	Open-source, general-purpose policy engine that decouples policy decision-making from policy enforcement. A service supplies structured data (JSON) as input; OPA evaluates it against policies and data and returns a decision. Graduated project of the Cloud Native Computing Foundation (CNCF).	[12], [13]
Rego	OPA's declarative policy language. Inspired by Datalog and extended to structured documents such as JSON; a policy is a set of rules that define values from the `input` document and loaded `data`.	[12], [14]
LegalRuleML	OASIS standard XML vocabulary for representing legal norms, including deontic operators, defeasibility and links between rules and their legal sources.	[6], [7]
Decision table	Tabular specification listing combinations of conditions and the resulting action. Used for "decision-table testing": one test per row.	[23]
Three-valued logic	Logic with truth values true, false and unknown (Kleene). InspectR uses it so that missing evidence is not forced into true or false.	[15]
Fail-safe defaults	Security design principle: base decisions on explicit permission, so that the default on missing information is denial. InspectR's "fail-closed" rule applies it to evidence.	[16]
DPIA	Data Protection Impact Assessment: assessment the controller must carry out before processing that is likely to result in a high risk, including large-scale processing of special categories of data (GDPR Art. 35(1), 35(3)(b); special categories in Art. 9).	[1]

3. What is Policy-as-Code, and how does it work?

Definition. Policy-as-Code (PaC) means writing a policy as code in a dedicated policy language, storing it in version control, testing it automatically, and letting a policy engine evaluate it against machine-readable input [12]. The output is a structured decision (for example `allow: true`, a list of violations, or in InspectR a status with evidence) rather than a human reading a document.

Core idea: separate the decision from enforcement. OPA documents this as decoupling: the system that needs a decision (an API gateway, a Kubernetes admission controller, a CI pipeline,

or InspectR's Python evaluator) sends a query with JSON input; OPA evaluates the policy and returns the decision; the calling system decides what to do with it [12].

```
             +---------------------+
  input ---> |  Policy engine (OPA) | ---> decision (JSON)
  (JSON)     |  policy (Rego)       |      e.g. {"status": "FAIL", ...}
             |  data   (JSON)       |
             +---------------------+
```

How it works, step by step.

1. Author. The policy is written in a policy language (Rego for OPA).
2. Version. Policies live in Git: changes are reviewed, diffed and traceable.
3. Test. Unit tests for the policy run automatically (`opa test`) [12].
4. Evaluate. The engine receives `input` (here: the pipeline blueprint), evaluates the rules, and returns a decision (`opa eval`, the REST API, or an embedded library).
5. Act. The caller enforces or reports the decision. InspectR only reports; it does not block deployment.

Where PaC is used. Mainly infrastructure-as-code, CI/CD and Kubernetes admission control. Examples of PaC tools: OPA/Rego (general purpose), Conftest (Rego for configuration files), Kyverno (Kubernetes policies in YAML), HashiCorp Sentinel (Terraform), and AWS Cedar (authorisation). InspectR applies PaC to a different target: the design of a data pipeline, before it is built.

What PaC does not do. PaC executes an interpretation of a policy. It cannot decide open legal questions, and it is only as good as the input facts. This is why InspectR has the statuses HUMAN (judgement needed) and UNKNOWN (facts missing).

4. OPA and Rego examples

All examples below are verified with OPA v1.4.2 (`opa check --strict`, `opa test`, `opa eval`). They use Rego v1 syntax (`if`, `contains`, `every`), which is the default since OPA 1.0; `import rego.v1` keeps the files compatible with older OPA versions [12].

4.1 Minimal policy: allow or deny

```
package example.storage

import rego.v1

default allow := false

allow if {
    input.bucket.encrypted == true
    input.bucket.public == false
}
```

Expressions inside a rule body are joined by AND; `default` gives the value when no rule body matches. For the input `{"bucket": {"encrypted": true, "public": true}}`:

```
$ opa eval -d storage.rego -i bucket.json 'data.example.storage.allow'
false
```

4.2 InspectR-style rule: a GDPR requirement as a decision table

A simplified version of rule TK-R1 (the full rule is `policies/tk/TK-R1.rego`). Requirement: if a pipeline processes special-category data (GDPR Art. 9), the design must contain a DPIA checkpoint (GDPR Art. 35).

```
package example.dpia

import rego.v1

nodes := object.get(object.get(input, "pipeline", {}), "nodes", [])

sources := [n | some n in nodes; n.type == "data_source"]

gates := [n | some n in nodes; n.type == "compliance_gate"]

# Trigger: GDPR Art. 9 special-category data is processed
trigger_true if {
    some s in sources
    s.properties.contains_special_category_data == true
}

trigger_false if {
    count(sources) > 0
    every s in sources {
        s.properties.contains_special_category_data == false
    }
}

# Control: a DPIA checkpoint is declared (GDPR Art. 35)
control_true if {
    some g in gates
    g.properties.dpia_checkpoint_defined == true
}

control_false if {
    some g in gates
    g.properties.dpia_checkpoint_defined == false
}

control_false if count(gates) == 0

# Decision table: first matching row wins; fall-through is fail-closed
status := "NOT_APPLICABLE" if {
    trigger_false
} else := "FAIL" if {
    trigger_true
    control_false
} else := "PASS" if {
    trigger_true
    control_true
} else := "UNKNOWN"
```

Key points: comprehensions (`[n | some n in nodes; ...]`) select nodes; `some` means “there exists”, `every` means “for all”; the `else` chain encodes the decision table in order; the final `else := "UNKNOWN"` makes the rule fail-closed, so a missing or non-boolean field can never produce PASS.

Evaluation on a small blueprint:

```
$ cat input.json
{"pipeline": {"nodes": [
```

```
  {"id": "health_records", "type": "data_source",
   "properties": {"contains_special_category_data": true}},
  {"id": "dpia_check", "type": "compliance_gate",
   "properties": {"dpia_checkpoint_defined": true}}
]}}

$ opa eval -d dpia.rego -i input.json 'data.example.dpia.status'
"PASS"
```

4.3 Testing a policy with `opa test`

Tests are Rego rules whose names start with `test_`; `with input as ...` replaces the input. One test per decision-table row:

```
package example.dpia_test

import data.example.dpia
import rego.v1

src(v) := {"id": "s1", "type": "data_source",
    "properties": {"contains_special_category_data": v}}

gate(v) := {"id": "g1", "type": "compliance_gate",
    "properties": {"dpia_checkpoint_defined": v}}

test_pass if dpia.status == "PASS"
    with input as {"pipeline": {"nodes": [src(true), gate(true)]}}

test_fail if dpia.status == "FAIL"
    with input as {"pipeline": {"nodes": [src(true), gate(false)]}}

test_not_applicable if dpia.status == "NOT_APPLICABLE"
    with input as {"pipeline": {"nodes": [src(false), gate(true)]}}

test_missing_trigger_is_unknown if dpia.status == "UNKNOWN"
    with input as {"pipeline": {"nodes": [
        {"id": "s1", "type": "data_source", "properties": {}},
        gate(true),
    ]}}
```

```
$ opa test . -v
data.example.dpia_test.test_pass: PASS
data.example.dpia_test.test_fail: PASS
data.example.dpia_test.test_not_applicable: PASS
data.example.dpia_test.test_missing_trigger_is_unknown: PASS
PASS: 4/4
```

The InspectR rule set itself is checked the same way: 29 Rego unit tests cover every decision-table row of TK-R1–TK-R3 (`opa test policies/tk -v`).

5. How to translate regulation into code

Legal text is written for human interpretation; a rule engine needs exact conditions and typed facts. The translation is therefore an act of interpretation that must be documented and reviewable. InspectR follows these steps, grounded in requirements-engineering and legal-informatics work on regulatory rules [5], [6], [8], [9]:

Step	What is done	Output in InspectR
1. Select the provision	Identify the legal or organisational source and its exact text.	Requirement with legal reference (e.g. GDPR Art. 35(1), 35(3)(b), Art. 9)
2. Identify the normative structure	Find the modality (obligation, permission, prohibition), the addressee, the condition and the required action or state [6], [8].	Trigger and obligation
3. Separate checkable from judgement elements	Vague or open-textured terms (“high risk”, “large scale”, “compatible purpose”) need human judgement [10]. Do not encode them as if they were exact.	Judgement rules return HUMAN, never PASS (TK-R3)
4. Define the facts	Decide which typed blueprint fields represent trigger and control, and whether tooling can export them or they must be annotated manually.	Facts with three values: true, false, undetermined
5. Write the decision table	List every combination of trigger and control values and assign exactly one status; missing evidence goes to UNKNOWN.	Complete, exclusive decision table [23]
6. Encode	One rule per requirement, kept structurally close to its source (isomorphism) so it can be traced and maintained [9].	One Rego package per rule, result with reason, evidence, legal reference and remedy
7. Test	One test per decision-table row, plus scenarios that change one fact at a time.	`opa test`; scenarios S0–S6
8. Review and version	Domain or legal experts check that the rule matches the intended interpretation; changes are versioned.	Rule set version (`tk-v2.0`); expert review is open (AC7)

Worked example (TK-R1).

- Source: GDPR Art. 35(1) requires a DPIA before processing likely to result in a high risk; Art. 35(3)(b) names large-scale processing of special categories of data (Art. 9) as a case requiring one [1]. TK lists “DPIA / risk assessment” as a requirement.
- Rule: IF a data source declares `contains_special_category_data: true` THEN one compliance gate declares `dpia_checkpoint_defined: true` and `risk_assessment_checkpoint_defined: true`.
- Interpretation recorded: “large scale” and “high risk” are not evaluated by the rule; the trigger is a deliberate simplification, documented as such.
- Result: the design either declares the checkpoints (PASS), declares them false or has no gate (FAIL), lacks the facts (UNKNOWN), or has no special-category data (NOT_APPLICABLE). PASS means the design declares a DPIA checkpoint, not that a DPIA was carried out.

Limits to state openly. Encoded rules are interpretations, not the law; legal references are traceability links, not validated legal mappings; a blueprint can declare a control that is never implemented; and judgement-dependent requirements cannot be reduced to PASS/FAIL. These limits are the reason for the assurance boundary above.

References

1. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation). Official Journal of the European Union, L 119, 4.5.2016, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj
2. European Data Protection Board (2020). Guidelines 4/2019 on Article 25: Data Protection by Design and by Default, Version 2.0, adopted 20 October 2020.
3. Cavoukian, A. (2009). Privacy by Design: The 7 Foundational Principles. Information and Privacy Commissioner of Ontario, Canada.
4. Sadiq, S., Governatori, G., & Namiri, K. (2007). Modeling control objectives for business process compliance. In Business Process Management (BPM 2007), LNCS 4714, pp. 149–164. Springer.
5. Governatori, G., & Sadiq, S. (2009). The journey to business process compliance. In J. Cardoso & W. van der Aalst (Eds.), Handbook of Research on Business Process Modeling, pp. 426–454. IGI Global.
6. OASIS (2021). LegalRuleML Core Specification Version 1.0. OASIS Standard.
7. Palmirani, M., Governatori, G., Rotolo, A., Tabet, S., Boley, H., & Paschke, A. (2011). LegalRuleML: XML-based rules and norms. In Rule-Based Modeling and Computing on the Semantic Web (RuleML 2011 – America), LNCS 7018, pp. 298–312. Springer.
8. Breaux, T. D., & Antón, A. I. (2008). Analyzing regulatory rules for privacy and security requirements. IEEE Transactions on Software Engineering, 34(1), 5–20.
9. Bench-Capon, T. J. M., & Coenen, F. P. (1992). Isomorphism and legal knowledge based systems. Artificial Intelligence and Law, 1(1), 65–86.
10. Hart, H. L. A. (1961). The Concept of Law. Oxford: Clarendon Press (chapter VII, “open texture”).
11. Mohun, J., & Roberts, A. (2020). Cracking the Code: Rulemaking for Humans and Machines. OECD Working Papers on Public Governance, No. 42. OECD Publishing, Paris.
12. Open Policy Agent. OPA Documentation (Introduction, Policy Language, Policy Testing). https://www.openpolicyagent.org/docs (accessed November 2026).
13. Cloud Native Computing Foundation. Open Policy Agent (OPA) project page. https://www.cncf.io/projects/open-policy-agent-opa/
14. Ceri, S., Gottlob, G., & Tanca, L. (1989). What you always wanted to know about Datalog (and never dared to ask). IEEE Transactions on Knowledge and Data Engineering, 1(1), 146–166.
15. Kleene, S. C. (1952). Introduction to Metamathematics. Amsterdam: North-Holland.
16. Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278–1308.
17. ISO/IEC 27000:2018. Information technology — Security techniques — Information security management systems — Overview and vocabulary. ISO.
18. NIST (2020). Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5. https://doi.org/10.6028/NIST.SP.800-53r5
19. OASIS (2020). Static Analysis Results Interchange Format (SARIF) Version 2.1.0. OASIS Standard. https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html
20. Amazon Web Services. AWS Config API Reference: Compliance (ComplianceType). https://docs.aws.amazon.com/config/latest/APIReference/API_Compliance.html
21. Kyverno. Policy Reports. https://kyverno.io/docs/policy-reports/
22. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75–105.
23. ISTQB. Standard Glossary of Terms Used in Software Testing: “decision table testing”. https://glossary.istqb.org